Systematic Mutation and Black-Box Probing for SQL Injection WAF Bypass: A Live Testbed and Family-Level Analysis

Tran Xuan Quyen
School of Info. & Comm. Technology
HUST
Hanoi, Vietnam
quyen.tx252593m@sis.hust.edu.vn

Co Huy Dung
School of Info. & Comm. Technology
HUST
Hanoi, Vietnam
dung.ch252592m@sis.hust.edu.vn

Nguyen Duc Toan
School of Info. & Comm. Technology
HUST
Hanoi, Vietnam
toan.nguyenduc1@hust.edu.vn

***Abstract*—Attackers routinely evade Web Application Firewalls (WAFs) by mutating SQL injection (SQLi) payloads. Prior studies report high bypass rates, yet these aggregate figures often conflate heterogeneous operators and depend on search biases. Consequently, defenders need reproducible, family-resolved measurements tied to explicit labels.**

This paper presents an offensive methodology that generates discrete SQLi variants through a dialect seed, deployment-specific augmentations, and learned transform rules. We issued black-box HTTP probes against a live AWS WAF-fronted application, collecting per-attempt JSONL logs. In our 22,230-variant deployment experiment, our operational classifier identified a 47.7% clean bypass rate (HTTP 200 OK), with an additional 30.2% of attempts inducing server-side errors (HTTP 500) that confirm database-layer reach, yielding a total WAF evasion rate of 77.9%. Family-level analysis reveals extreme heterogeneity: double URL-encoding and null-byte cohorts achieved bypass rates exceeding 99.9%, while space-replacement, tab/newline, and comment-block mutations were systematically suppressed (81.2%–92.9% blocked). An independent ground-truth audit across sampled responses verified 100.0% classification precision.

As a secondary control, we evaluated repeated RFC percent-decoding against a TF-IDF logistic-regression surrogate on a 20-sample pilot (13 train / 7 test). The surrogate achieved an accuracy of 3/7 (42.9%) on both raw and canonicalized inputs, suggesting on this pilot scale that naive decoding does not immediately rescue simple linear surrogate models. All reported rates represent single-session point estimates for this concrete deployment instance; the core contribution lies in the explicit methodology, decomposed statistics, and reproducible artifact.

***Index Terms*—SQL injection, web application firewall, black-box attack, adversarial mutation, bypass measurement, variant families**

I. Introduction

Web applications remain the primary interface for online services, with Web Application Firewalls (WAFs) widely deployed to block injection attacks [9]. From an attacker's perspective, the key challenge is navigating a specific deployment's concrete rules and parsers to achieve a security-relevant application outcome. Signatures can be evaded by semantic-preserving rewrites, and parsers can disagree on decoding and parameter binding. While these phenomena are well known, measuring the conditions and specific mutation structures under which bypasses occur remains a scientific challenge.

This paper contributes an operational, attack-centric evaluation. We systematically mutate SQLi fragments, probe a production-style AWS WAF deployment, and analyze outcomes at both aggregate and variant-family levels. We release our implementation as a reproducible artifact to provide red and blue teams with empirical distributions for prioritization and regression testing.

A. Problem Statement

SQLi evasion combines representation-level mutation—altering substrings or encodings seen by a pattern matcher—and interpretation-level divergence, where the WAF and application decode the same sequence differently [2].

Prior adversarial methods demonstrate that the attack space is combinatorial. Systems like SSQLi [1], GSQLi [7], and WAF-A-MoLE [6] achieve high bypass rates by searching this space. However, success is rarely uniform across operators. Published rates seldom decompose by operator family on live managed WAFs, obscuring which mutations survive. Furthermore, while mutating HTTP structure induces parsing discrepancies [2], we focus on URL-visible injection with deterministic seed remapping. This minimizes parameter-name mismatch, providing a controlled setting to study representation-level evasion against a fixed policy.

B. Objectives and Research Questions

We structure our empirical evaluation around three core questions:

- **RQ1 (Scale and semantics).** Under an explicit bypass definition, what fraction of attempts succeed against our AWS WAF lab, and how does this relate to unique payload cardinality?
- **RQ2 (Structure).** How much bypass mass concentrates in specific mutation families, and which are systematically suppressed?
- **RQ3 (Defense and Normalization).** To what extent can standard RFC percent-decoding or surrogate machine

learning models counter these representation-level evasions, and does input canonicalization restore detector precision?

In addressing these questions, we formalize a black-box attack pipeline for managed WAF assessment. We provide an empirical characterization of a live AWS WAF run, detailing unique payload counts and family-level bypass rates that reveal sharp heterogeneity. Finally, we include a secondary control showing that basic canonicalization does not immediately rescue simple classifiers.

C. Paper Organization

Section II covers related work. Section III details our methodology. Section IV presents the evaluation answering RQ1–RQ3. Section V concludes the paper.

II. Background and Related Work

Offensive SQLi evaluation bridges adversarial search against detectors, HTTP semantics, and operational security measurement. We highlight strands that motivate our focus on family-resolved reporting.

A. Mutation-Based SQLi Attacks

Adversarial methods search for inputs that preserve malicious intent while evading detectors. Reinforcement learning approaches like SSQLi [1], GAN-based methods like GSQLi [7], and semantics-preserving fuzzing like WAF-A-MoLE [6] achieve high bypass rates. The underlying attack space is combinatorial, involving comment insertions, encodings, and dialect-specific spellings that interact non-monotonically with tokenized defenses. Consequently, the relevant metric is not just the aggregate bypass rate, but which specific compositions survive, as this informs both attacker and defender strategies.

B. Parsing Discrepancies and Defense Evaluation

When components disagree on decoding order or structural boundaries, attackers can satisfy edge predicates while delivering malicious semantics [2]. WAFFLED demonstrated this across major WAFs. Our experiments stabilize parameter binding via remapped seeds to isolate representation-level variance.

Hybrid defenses aim to raise evasion costs through better data [8], shadow models [5], or preprocessing [4]. However, evaluating these defenses requires paired offensive benchmarks. Without logs tying attempts to mutation families, it is difficult to determine if a mitigation removes high-yield operators or merely reshuffles the search order. Our work supplies a log-grounded baseline for such comparisons.

C. Metrics and Confounding Threats

Black-box evaluation relies on observable HTTP outcomes. Since different probes map the same vulnerability to different observables, any operational definition of a bypass embeds epistemic commitments. Transparent scripting and published logs mitigate this issue, though they do not eliminate it entirely.

Seeds → augmentations & rules → 2nd-layer transforms → probes → logs

Fig. 1. Offensive evaluation workflow; policies configurable in `config.json`.

III. Methodology

This section details our adversarial evaluation methodology, including the experimental environment, adversary capabilities, and the attack pipeline. Artifacts are available under `paper/experiment`.

A. Experimental Environment

We deployed the study on AWS in the Asia Pacific (Sydney) region (`ap-southeast-2`), routing adversarial traffic through an AWS Application Load Balancer (ALB) integrated with AWS WAF before reaching an EC2-hosted application. This mirrors an enterprise cloud deployment where the backend only sees what the edge WAF forwards.

The Web ACL (`test-waf`) features three sequential rules:

1) **`Block-SQLi-Strict-Pattern`:** Targets simple tautologies (e.g., `1=1`) evaluated across all query arguments after decoding and normalization.
2) **`AWS-AWSManagedRulesSQLiRuleSet`:** Amazon's managed SQLi bundle, set to block any hit, providing broad vendor coverage across the request structure.
3) **`Final-Optimized-SQLi-Rule`:** A custom regular expression stage targeting specific lab leftovers, such as hex-flavored encodings and null-byte obfuscation.

The backend application is a production-style NestJS REST service backed by a PostgreSQL database on an EC2 instance. We drove probes programmatically via custom asynchronous HTTP client scripts without relying on generic commercial scanners like sqlmap. Outcomes were observed directly from HTTP responses: a `403` indicated a WAF/ALB edge block, a `200` confirmed clean bypass with data leakage, a `500` confirmed database-level unhandled exception, and a `404` indicated application-layer path reach without query execution.

B. Adversary Model and Attack Pipeline

We model a network adversary capable of sending HTTP/1.1 requests, observing responses, and adapting mutations based on logs. The adversary lacks white-box access, aligning with typical red-team constraints [1], [6].

The end-to-end pipeline operates as follows:

$$\begin{aligned} \mathcal{M} : \text{Seeds} \to \text{Augmentations} \to \text{Transforms} \to \mathcal{V}, \\ \mathcal{V} \xrightarrow{\text{HTTP}} \text{WAF} \to \text{App} \to \text{Label}. \end{aligned} \tag{1}$$

The driver serializes each variant in $\mathcal{V}$ as a GET request, logs the response, and outputs a JSON object.

C. Staged Mutation Generation

To balance diversity and reproducibility, generation is staged:

- **Seeds:** We use PostgreSQL- and Oracle-oriented templates, affecting syntax and tokenization. Numeric seed

tokens are remapped to stable string prefixes to align WAF and application parameter tracking.

- **Learned Rules:** The harness injects deployment-specific variants, biasing the search towards historically informative outcomes. Prior to commencing official data collection, the mutation policy state was strictly purged (via the `--fresh` flag), resetting the operator exclusion list to empty (`exclude: []`) to prevent survivorship bias from earlier test probes.
- **Transforms:** An optional second layer applies further evasion transforms subject to configuration limits. Consequently, the empirical distribution reflects practical search limits rather than an unconstrained draw from all possible SQLi strings.

D. Outcome Semantics and Metrics

Each attempt i yields an HTTP status s_i and body b_i, and is labeled $y_i \in \{clean_bypass, error_bypass, blocked, other\}$. A blocked label denotes edge denial. A bypass label indicates a WAF-pass with application-visible signals consistent with effective SQLi side effects.

An attempt i is considered as $y_i = \text{clean_bypass}$ if the HTTP response status code is 200 OK, indicating a clean bypass reaching the application logic with returned entity records; $y_i = \text{error_bypass}$ if the status code is 500 Internal Server Error, indicating that the mutated payload successfully bypassed the WAF edge and caused an unhandled PostgreSQL driver syntax exception at the database layer. An attempt is labeled $y_i = \text{blocked}$ if intercepted at the perimeter ($s_i = 403$), and $y_i = \text{other}$ if the request passed the WAF to the application router but returned 404 Not Found (e.g., altered project query strings).

Although our tracking harness accounts for client validation errors (HTTP 400/422) as potential application-reach indicators, the target NestJS route accepts string query arguments without schema type-rejections. As verified in Section IV, zero 400 or 422 responses occurred ($N_{400/422} = 0$), meaning all successful WAF evasions cleanly partitioned into either 200 OK or 500 Server Errors.

Let N be the total attempts and N_{bp} the bypass count ($N_{200} + N_{500}$). The aggregate bypass rate is:

$$\text{BR}_{\text{agg}} = \frac{N_{\text{bp}}}{N} \tag{2}$$

For a mutation family f, the family-level bypass rate is $\text{BR}_f = N_{f,\text{bp}}/N_f$.

E. Secondary Control

To assess normalization effects, we applied fixpoint `unquote_plus` decoding to probe strings and trained a TF-IDF logistic-regression surrogate (`ml_baseline.py`) on a 20-sample labeled pilot dataset partitioned into 13 training and 7 test instances (35% test split, random seed 42). This pilot tests whether minimal RFC-relevant decoding aligns feature spaces sufficiently to improve a simple linear classifier.

IV. Experiments and Results

We answered RQ1–RQ3 using empirical data exported from live AWS WAF probing and an RFC decoding surrogate pilot. Probing scripts and analysis pipelines were executed in a Windows Python 3.12 environment using standardized data science and machine learning libraries.

A. Experimental Flow

We followed a consistent adversarial measurement workflow:

1) Initialize baseline SQLi seeds with dialect-specific syntax (PostgreSQL/Oracle) and remap parameter IDs to match target endpoint routing.
2) Dynamically mutate seeds across staged transformation families and dispatch black-box HTTP GET requests to the live AWS ALB endpoint.
3) Classify HTTP responses into decomposed semantic outcomes (`clean_bypass`, `error_bypass`, `blocked`, `other`) via verified response body structural inspection.
4) Aggregate outcomes across transformation families and evolutionary generations to quantify evasion gains and policy suppression.
5) Conduct an independent, non-circular ground-truth audit on randomly sampled responses to validate label precision.

B. Overall Outcome Distribution

Table I presents the high-level outcome distribution across the complete live dataset of $N = 22{,}230$ tested payload variants.

TABLE I
Overall result distribution across $N = 22{,}230$ live probes

Result Classification	Count	Percentage
Clean Exploit Bypass (200 OK data leak)	10,599	47.7%
Database Error Bypass (500 SQL syntax error)	6,710	30.2%
Total Effective Evasion (Passed WAF)	**17,309**	**77.9%**
Blocked by AWS WAF / Edge (403 Forbidden)	3,633	16.3%
Application Other (404 Not Found)	1,288	5.8%
Total Probes Evaluated	**22,230**	**100.0%**

C. Status Code Breakdown

Table II details the distribution of observed HTTP response status codes.

D. Transformation Effectiveness

Table III summarizes the status code distribution and empirical bypass/block rates decomposed across all evaluated transformation families.

TABLE II
HTTP STATUS DISTRIBUTION AND SEMANTIC INTERPRETATIONS

Status Code	Count	%	Operational Interpretation
200 OK	10,599	47.7%	Verified data leakage; backend returned valid project entity JSON.
403 Forbidden	3,633	16.3%	Request denied by AWS WAF ruleset or ALB edge filter.
404 Not Found	1,288	5.8%	Reached application router; payload altered query parameter key/value without triggering injection.
500 Server Error	6,710	30.2%	Unhandled PostgreSQL syntax/type exception proving WAF pass to DB layer.
Total	**22,230**	**100.0%**	Complete closed-form sum ($N_{400/422} = 0$, $N_{502} = 0$).

E. RQ1: Scale, Label Precision, and Evolutionary Gain

Our empirical testbed evaluated $N = 22{,}230$ variants against the live AWS WAF deployment. In response to RQ1:

1) **Aggregate Evasion Scale:** As shown in Table I, 17,309 out of 22,230 attempts (77.9%) successfully evaded the WAF edge rulesets. Of these, 10,599 attempts (47.7%) achieved clean exploitation (HTTP 200), extracting valid backend project entities, while 6,710 attempts (30.2%) bypassed the edge to cause unhandled database-layer exceptions (HTTP 500). Only 3,633 attempts (16.3%) were intercepted at the perimeter. The remaining 1,288 attempts (5.8%) bypassed the edge into the application router but matched no project entity (HTTP 404). Crucially, zero requests returned HTTP 400 or 422 ($N_{400/422} = 0$), as the NestJS endpoint route accepts unconstrained query strings without schema rejection.
2) **Ground-Truth Precision Audit:** To verify classification validity without circular reliance on classifier regexes, we conducted an independent structural audit on $N_s = 400$ randomly sampled responses (100 per outcome category). The audit verified 100.0% precision across all categories:
   - 200 OK responses matched the complete NestJS Entity JSON schema (`projectName`, `id`, `applicationDomain`).
   - 500 responses strictly matched native PostgreSQL driver syntax exception signatures.
   - 403 responses exhibited AWS ALB/WAF error markers (`awselb/2.0`, `403 Forbidden`).
   - 404 responses satisfied NestJS exception envelope schemas.
3) **Evolutionary Gain over Baseline:** Prior to execution, the transform exclusion list was reset (`--fresh`) to guarantee a clean initial state without survivorship bias. Comparing pure unadapted initial seeds (Generation 0, $N_0 = 97$) against subsequent adaptive mutations demonstrates significant evolutionary efficacy:
   - Gen 0 Baseline: 200 OK rate was 7.2% (7/97), total bypass rate was 11.3% (11/97), and block rate was 85.6% (83/97).
   - Adaptive Generations: 200 OK rate surged to 47.9% (10,592/22,133), and total evasion reached 78.2% (17,298/22,133).
   - This constitutes an empirical **Evolutionary Gain of** +40.7% for clean exploit yield and +66.9% for perimeter evasion, confirming that adaptive multi-layered mutations systematically discover WAF parsing blind spots.

F. RQ2: Family-Level Heterogeneity

In addressing RQ2, our results reveal severe polarization in evasion efficacy across mutation operators, rather than a uniform distribution of vulnerability:

- **Dominant Evasion Families:** The `double_url_encode` family ($N_f = 7{,}683$) and `null_byte` family ($N_f = 7{,}837$) achieved near-total WAF perimeter evasion, each exhibiting a total bypass rate of 99.9% ($> 61\%$ clean 200 OK and $> 38\%$ database 500 error). AWS WAF's normalization pipeline fails to collapse multi-layer percent encodings prior to rule evaluation, yet the downstream NestJS/Node.js URI parser transparently unpacks them, delivering malicious SQL syntax to the database driver.
- **The Semantic Evasion Fallacy in `case_mix`:** In Table III, the `case_mix` family ($N_f = 1{,}454$) exhibits a remarkably low block rate of 0.3% (5/1,454), which superficially suggests high evasion capability. However, its clean exploit bypass rate is only 4.2% (61/1,454); instead, 86.4% of attempts (1,257/1,454) resulted in HTTP 404 Not Found. Because case-mixing mutates SQL keywords as well as entity literals, the altered project identifier fails to match database records under PostgreSQL's case-sensitive collation. This reveals that perimeter WAF evasion does not imply functional exploitability: inputs can easily bypass edge pattern filters while simultaneously nullifying application-level attack objectives.
- **Systematically Suppressed Families:** In stark contrast, structural whitespace modifiers and SQL comments are heavily mitigated by AWS WAF's lexical tokenization:
  - `comment_slash` (`/**/`): 92.9% blocked (13/14).
  - `comment_inline` (`/**/`): 84.6% blocked (11/13).
  - `tab_newline` (`\t`, `\n`): 81.2% blocked (13/16).
  - `url_encode` (standard single percent-encoding): 68.9% blocked (3,548/5,148).

This stark dichotomy proves that AWS WAF Managed Rulesets are heavily tuned for syntactic token boundary disruptions, but possess architectural blind spots against encoding-depth discrepancies and string termination anomalies.

TABLE III
HTTP STATUS CODE DISTRIBUTION BY TRANSFORM TYPE ACROSS $N = 22{,}230$ VARIANTS

Transform Family	200 OK	403 Block	404	500 Err	Total (N_f)	Clean Bypass%	404 Rate%	Block Rate%
null_byte	4,805	4	2	3,026	7,837	61.3%	0.0%	0.1%
double_url_encode	4,743	6	0	2,934	7,683	61.7%	0.0%	0.1%
url_encode	984	3,548	2	614	5,148	19.1%	0.0%	68.9%
case_mix	61	5	1,257	131	1,454	4.2%	86.4%	0.3%
hex	4	7	27	0	38	10.5%	71.1%	18.4%
tab_newline	1	13	0	2	16	6.2%	0.0%	81.2%
comment_slash	0	13	0	1	14	0.0%	0.0%	92.9%
comment_inline	0	11	0	2	13	0.0%	0.0%	84.6%
other_transform	0	6	0	0	6	0.0%	0.0%	100.0%
triple_url_encode*	1	0	0	0	1	100.0%	0.0%	0.0%
original / baseline*	0	1	0	0	1	0.0%	0.0%	100.0%
unclassified	0	19	0	0	19	0.0%	0.0%	100.0%
GRAND TOTAL	**10,599**	**3,633**	**1,288**	**6,710**	**22,230**	**47.7%**	**5.8%**	**16.3%**

Note. Clean Bypass% reflects HTTP 200 OK data leakage rate. Block Rate% reflects HTTP 403 edge denial rate. The total effective WAF evasion rate (200 OK + 500 Server Error) reaches 99.9% for double_url_encode and null_byte.

**Statistical Caveat.* Families with $N_f < 5$ (e.g., `triple_url_encode`, `original/baseline`) are included to preserve the closed-form sum of $N = 22{,}230$, but their apparent extreme rates (100% or 0%) are excluded from rate generalizations due to minute sample support.

G. RQ3: Normalization and Surrogate Defense Effectiveness

To investigate RQ3, we evaluated whether standard input normalization or surrogate machine learning filters could effectively mitigate the observed evasions.

- **RFC Normalization Control:** We evaluated repeated RFC percent-decoding (`urllib.parse.unquote_plus` fixpoint) on probe inputs. While naive single-layer decoding strips standard single encodings, fixpoint canonicalization often introduces syntactic anomalies or character truncations when confronted with compound null-bytes (`%00`).
- **Offline Classifier Surrogate:** We evaluated an offline TF-IDF logistic-regression surrogate classifier trained on canonicalized templates across a 20-sample pilot dataset partitioned into 13 training and 7 testing instances (35% test split, random seed 42). The classifier correctly predicted only 3 out of 7 test instances (42.9% accuracy) for both raw and canonicalized inputs. Because adversarial mutations simultaneously preserve SQL injection intent while mimicking benign token distributions, simple linear or token-frequency models fail to distinguish benign encoding density from evasion payloads on this pilot scale.

These results confirm that naive pre-filtering cannot unilaterally rescue edge firewalls without semantic-aware, AST-level deep inspection.

H. Threats to Validity and Reproducibility

Upstream Gateway Denial-of-Service Side Effects. During preliminary exploratory probing, unencoded raw binary null-byte sequences (`\x00`) caused intermittent 502 Bad Gateway responses from the AWS ALB, indicating upstream worker crashes in the runtime C-bindings of the database driver or Node.js runtime. For our official experiment ($N = 22{,}230$), raw binary bytes were sanitized to standard RFC percent-encodings (`%00`) and coupled with exponential-jitter retry policies, ensuring all recorded outcomes reflect stable application processing ($N_{502} = 0$). However, this observation highlights an orthogonal availability-attack vector: string termination anomalies can induce denial-of-service side effects rather than confidentiality breaches.

Single-Session Point Estimation. All reported metrics represent an empirical point estimate from a continuous, large-scale execution session ($N = 22{,}230$). While session stability and precision were verified via periodic calibration drift checks and ground-truth audits, multi-session confidence intervals were not computed due to operational and AWS cloud infrastructure cost constraints. Consequently, the rates describe this concrete deployment instance rather than a multi-trial population distribution.

Scope and Reproducibility. Our findings are bounded by a single AWS WAF policy configuration, an HTTP GET query-parameter attack vector, and a PostgreSQL backend. All probe scripts, variant generators, audit harnesses, and raw JSONL logs ($N = 22{,}230$) are published in the project artifact repository. Probing was conducted exclusively against authorized cloud testbeds.

V. DISCUSSION AND CONCLUSION

This work decomposes SQLi WAF bypass analysis along semantics, scale, and structure under an empirical, live-environment search distribution.

A. Research Question Answers

RQ1 (Scale and semantics): Our comprehensive live execution across $N = 22{,}230$ payload variants revealed an aggregate WAF perimeter evasion rate of 77.9% ($17{,}309/22{,}230$), decomposing into a 47.7% clean exploit bypass rate (HTTP 200 data leakage) and a 30.2% database-layer error reach rate (HTTP 500), with zero 400/422 validation rejections ($N_{400/422} = 0$). Crucially, adaptive mutation yielded an evolutionary gain of $+40.7\%$ in clean exploitation over unmutated Generation 0 baseline seeds (established without

survivorship bias), while an independent structural ground-truth audit confirmed 100.0% labeling precision across 400 sampled outcomes.

RQ2 (Structure): Evasion efficacy is intensely polarized across mutation categories. The `double_url_encode` ($N_f = 7{,}683$) and `null_byte` ($N_f = 7{,}837$) families each achieved a 99.9% edge bypass rate under the fixed AWS WAF managed ruleset. In contrast, structural space modifications (`tab_newline`: 81.2% blocked) and comment-block injections (`comment_inline`: 84.6% blocked; `comment_slash`: 92.9% blocked) were systematically suppressed. Furthermore, the `case_mix` family (86.4% 404 responses) demonstrates that edge-evasion does not guarantee application exploitation. This proves that AWS WAF's lexical tokenization is well-optimized for syntactic delimiter anomalies, but exhibits fundamental vulnerabilities to encoding-depth discrepancies.

RQ3 (Defense and Normalization): Offline canonicalization controls demonstrate that naive single- or multi-layer RFC percent-decoding fails to eliminate evasion risks, with a surrogate TF-IDF logistic-regression model achieving only 3 out of 7 correct predictions (42.9% accuracy) on test instances across both raw and canonicalized inputs. Effective defense necessitates semantic AST-level validation rather than superficial regex-based edge filtering.

B. Implications and Future Work

For defenders, WAF policy tuning and regression testing must be family-aware; relying on coarse aggregate dashboards obscures massive vulnerabilities in specific encoding vectors. Edge filters must be coupled with strict backend parameter binding and context-aware deserialization. Furthermore, our incidental discovery during exploratory testing of upstream 502 DoS errors induced by raw null-bytes demonstrates that adversarial mutation fuzzing can uncover availability vulnerabilities alongside confidentiality exploits. Future work should systematically evaluate cross-layer DoS resilience in managed cloud WAF architectures and extend multi-session variance measurement across geographically distributed deployments.

We presented a reproducible black-box methodology and empirical live-testbed analysis for managed WAF assessment. The core takeaway is that meaningful offensive measurement demands decomposed statistics and transparent search bias. Our family-level analysis identifies concrete operational blind spots, reinforcing the necessity of defense-in-depth security architectures.

Limitations and Ethics. All reported figures represent single-session point estimates bounded by our AWS WAF policy configuration and URL injection channel. All methodology was executed exclusively against authorized research infrastructure.

ACKNOWLEDGMENT

The authors thank the faculty and staff of the School of Information and Communication Technology, Hanoi University of Science and Technology, for their academic guidance and support throughout this research. The authors also appreciate the constructive feedback from colleagues and peers, which helped improve the quality of this work.

REFERENCES

[1] Y. Guan et al., "SSQLi: A Black-Box Adversarial Attack Method for SQL Injection Based on Reinforcement Learning," *Future Internet*, vol. 15, no. 4, p. 133, 2023.

[2] S. A. Akhavani et al., "WAFFLED: Exploiting Parsing Discrepancies to Bypass Web Application Firewalls," in *Proc. Security Research Workshop*, 2025.

[3] S. A. Akhavani et al., "HTTP-Normalizer: A robust proxy tool for RFC-compliant HTTP validation," in *Proc. Security Research Workshop*, 2025.

[4] B. Zhang et al., "BWAFSQLi: Bypassing Web Application Firewall with Adversarial SQL Injections," *ACM Trans. Softw. Eng. Methodol.*, 2026.

[5] C. Wu et al., "WAFBOOSTER: Automatic Boosting of WAF Security Against Mutated Malicious Payloads," in *Proc. Int. Conf. Cybersecurity*, 2025.

[6] L. Demetrio et al., "WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning," in *Proc. 35th ACM/SIGAPP Symp. Appl. Comput. (SAC)*, 2020.

[7] L. M. Khan et al., "GSQLi: A GAN-based Approach for Adversarial SQL Injection Sample Generation against WAF," in *Proc. Int. Conf. Inf. Secur.*, 2024.

[8] H. O. E. Elebiary et al., "Enhanced Web Payload Classification Using WAMM: An AI-Based Framework for Dataset Refinement and Model Evaluation," *J. Inf. Secur. Appl.*, 2026.

[9] OWASP, "Core Rule Set (CRS) Documentation," OWASP Foundation, 2020.

[10] R. Fielding et al., "RFC 7230: Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing," IETF, 2014.

[11] T. Berners-Lee et al., "RFC 3986: Uniform Resource Identifier (URI): Generic Syntax," IETF, 2005.